Package 'filearray'

October 11, 2026

Type Package

Title File-Backed Array for Out-of-Memory Computation

Version 0.2.4

Language en-US

Encoding UTF-8

License LGPL-3

URL https://dipterix.org/filearray/,
https://github.com/dipterix/filearray

BugReports https://github.com/dipterix/filearray/issues

Description Stores large arrays in files to avoid occupying large memories. Implemented with super fast gigabyte-level multi-threaded reading/writing via 'OpenMP'. Supports multiple non-character data types (double, float, complex, integer, logical, and raw).

Imports digest, fastmap (>= 1.1.1), methods, Rcpp, uuid (>= 1.1.0)

Suggests bit64, knitr, rmarkdown, testthat (>= 3.0.0)

LinkingTo BH, Rcpp

Config/testthat/edition 3

VignetteBuilder knitr

Config/roxygen2/version 8.1.0

NeedsCompilation yes

Author Zhengjia Wang [aut, cre, cph] (ORCID:
<https://orcid.org/0000-0001-5629-1116>)

Maintainer Zhengjia Wang <dipterix.wang@gmail.com>

Repository CRAN

Date/Publication 2026-10-11 17:00:02 UTC

Contents

apply . . . 2
as_filearray . . . 3
FileArray-class . . . 6
filearray_bind . . . 8
filearray_threads . . . 10
fmap . . . 10
fwhich . . . 13
mapreduce . . . 14
S3-filearray . . . 16
S4-filearray . . . 19
typeof . . . 30
with_filelock . . . 31

Index **33**

`apply` *Apply functions over file array margins (extended)*

Description

Apply functions over file array margins (extended)

Usage

```
apply(X, MARGIN, FUN, ..., simplify = TRUE)

## S4 method for signature 'FileArray'
apply(X, MARGIN, FUN, ..., simplify = TRUE)

## S4 method for signature 'FileArrayProxy'
apply(X, MARGIN, FUN, ..., simplify = TRUE)
```

Arguments

`X`	a file array
`MARGIN`	scalar giving the subscripts which the function will be applied over. Current implementation only allows margin size to be one
`FUN`	the function to be applied
`...`	optional arguments to `FUN`
`simplify`	a logical indicating whether results should be simplified if possible

Value

See Section 'Value' in `apply`;

as_filearray *Create or load existing file arrays*

Description

Create or load existing file arrays

Usage

```
as_filearray(x, ...)

as_filearrayproxy(x, ...)

filearray_create(
  filebase,
  dimension,
  type = c("double", "float", "integer", "logical", "raw", "complex"),
  partition_size = NA,
  initialize = FALSE,
  ...
)

filearray_load(filebase, mode = c("readwrite", "readonly"))

filearray_checkload(
  filebase,
  mode = c("readonly", "readwrite"),
  ...,
  symlink_ok = TRUE
)

filearray_load_or_create(
  filebase,
  dimension,
  on_missing = NULL,
  type = NA,
  ...,
  mode = c("readonly", "readwrite"),
  symlink_ok = TRUE,
  initialize = FALSE,
  partition_size = NA,
  verbose = FALSE,
  auto_set_headers = TRUE
)
```

Arguments

x	R object such as array, file array proxy, or character that can be transformed into file array
...	additional headers to check used by `filearray_checkload` (see 'Details'). This argument is ignored by `filearray_create`, reserved for future compatibility.
filebase	a directory path to store arrays in the local file system. When creating an array, the path must not exist.
dimension	dimension of the array, at least length of 2
type	storage type of the array; default is `'double'`. Other options include `'integer'`, `'logical'`, and `'raw'`.
partition_size	positive partition size for the last margin, or `NA` to automatically guess; see 'Details'.
initialize	whether to initialize partition files; default is false for performance considerations. However, if the array is dense, it is recommended to set to true
mode	whether allows writing to the file; choices are `'readwrite'` and `'readonly'`.
symlink_ok	whether arrays with symbolic-link partitions can pass the test; this is usually used on bound arrays with symbolic-links; see `filearray_bind`;
on_missing	function to handle file array (such as initialization) when a new array is created; must take only one argument, the array object
verbose	whether to print out some debug messages
auto_set_headers	whether to automatically set headers if array is missing or to be created; default is true

Details

The file arrays partition out-of-memory array objects and store them separately in local file systems. Since R stores matrices/arrays in column-major style, file array uses the slowest margin (the last margin) to slice the partitions. This helps to align the elements within the files with the corresponding memory order. An array with dimension `100x200x300x400` has 4 margins. The length of the last margin is 400, which is also the maximum number of potential partitions. The number of partitions are determined by the last margin size divided by `partition_size`. For example, if the partition size is 1, then there will be 400 partitions. If the partition size if 3, there will be 134 partitions. The default partition sizes are determined internally following these priorities:

1. the file size of each partition does not exceed `1GB`
2. the number of partitions do not exceed 100

These two rules are not hard requirements. The goal is to reduce the numbers of partitions as much as possible.

The arguments `...` in `filearray_checkload` should be named arguments that provide additional checks for the header information. The check will fail if at least one header is not identical. For example, if an array contains header key-signature pair, one can use `filearray_checkload(..., key = signature)` to validate the signature. Note the comparison will be rigid, meaning the storage type of the headers will be considered as well. If the signature stored in the array is an integer while provided is a double, then the check will result in failure.

Value

A FileArray-class instance.

Author(s)

Zhengjia Wang

Examples

```
# Prepare
library(filearray)
filebase <- tempfile()
if(file.exists(filebase)) { unlink(filebase, TRUE) }

# create array
x <- filearray_create(filebase, dimension = c(200, 30, 8))
print(x)

# Assign values
x[] <- rnorm(48000)

# Subset
x[1,2,]

# load existing array
filearray_load(filebase)

x$set_header("signature", "tom")
filearray_checkload(filebase, signature = "tom")

## Not run:
# Trying to load with wrong signature
filearray_checkload(filebase, signature = "jerry")

## End(Not run)


# check-load, and create a new array if fail
x <- filearray_load_or_create(
    filebase = filebase, dimension = c(200, 30, 8),
    verbose = FALSE, signature = "henry"
)
x$get_header("signature")

# check-load with initialization
x <- filearray_load_or_create(
    filebase = filebase,
    dimension = c(3, 4, 5),
    verbose = FALSE, mode = "readonly",
    on_missing = function(array) {
        array[] <- seq_len(60)
```

```
    }
)

x[1:3,1,1]

# Clean up
unlink(filebase, recursive = TRUE)
```

`FileArray-class` *Definition of file array*

Description

S4 class definition of `FileArray`. Please use `filearray_create` and `filearray_load` to create instances.

Public Methods

`get_header(key, default = NULL)` Get header information; returns `default` if `key` is missing

`set_header(key, value)` Set header information; the extra headers will be stored in meta file. Please do not store large headers as they will be loaded into memory frequently.

`can_write()` Whether the array data can be altered

`create(filebase, dimension, type = "double", partition_size = 1)` Create a file array instance

`delete(force = FALSE)` Remove array from local file system and reset

`dimension()` Get dimension vector

`dimnames(v)` Set/get dimension names

`element_size()` Internal storage: bytes per element

`fill_partition(part, value)` Fill a partition with given scalar

`get_partition(part, reshape = NULL)` Get partition data, and reshape (if not null) to desired dimension

`expand(n)` Expand array along the last margin; returns true if expanded; if the `dimnames` have been assigned prior to expansion, the last dimension names will be filled with `NA`

`initialize_partition()` Make sure a partition file exists; if not, create one and fill with `NA`s or 0 (`type='raw'`)

`load(filebase, mode = c("readwrite", "readonly"))` Load file array from existing directory

`lock_partition(parts, timeout = NULL)` Lock partitions `parts` (default: all) for this R session, so that other processes cannot write to them; waits and retries until all of them are locked or `timeout` seconds have passed (`NULL`: option `filearray.lock.timeout`, which defaults to waiting without limit; `0`: try once). Returns `TRUE` once locked, or `FALSE` after the time-out, with nothing locked; see section 'Partition locks'

`partition_path(part)` Get partition file path

`partition_size()` Get partition size; see `filearray`

`set_partition(part, value, ..., strict = TRUE)` Set partition value

`sexp_type()` Get data `SEXP` type; see R internal manuals

`show()` Print information

`type()` Get data type

`unlock_partition(parts)` Release one `lock_partition` lock on each of `parts`, or every `lock_partition` lock this R session holds on the array if `parts` is missing; returns `TRUE` if a lock was released

`valid()` Check if the array is valid.

Partition locks

Several processes (for example **parallel**, **callr** or **mirai** workers) can write to one array. Writes lock the partitions they change: sub-assignment (`x[...] <- value`), `fill_partition`, `initialize_partition`, `set_partition`, `fmap`, `fmap_element_wise` and `filearray_bind` wait while another process holds one of their partitions, so concurrent writers never lose each other's data. Reading takes no locks, so a reader may see a write that is still in progress. Functions that read but must first create missing partitions (`max`, `min`, `range` and `sum` with `na.rm = FALSE`, inputs of `fmap`, `filearray_bind`, evaluating array operations) lock the partitions they create, so they can wait too.

To keep partitions to yourself over several operations, for example to check whether a partition has been computed and write it only if not, lock them explicitly:

```
if (!x$lock_partition(k, timeout = 60)) stop("partition ", k, " is busy")
on.exit(x$unlock_partition(k), add = TRUE)
if (is.na(x[1, 1, k])) x[, , k] <- compute(k)
```

Locks belong to the R session, not to the object: every object of the same array shares them, writes of this session to partitions it holds do not wait, and each `lock_partition` call needs one `unlock_partition` call. Lock everything you need in one call: two sessions that each hold one partition and wait for the other's wait until their time-outs. Likewise, do not hold partitions that forked workers (`mclapply`) of the same session write. Non-interactive workers should set a time-out.

The locks are byte-range locks on an empty file `.filearray.lock` in the array directory (byte `k` locks partition `k`). They are advisory: older versions of this package and other programs ignore them. The operating system releases them when a process exits, so a crashed or killed process never leaves a stale lock. The locks guard writes but never make them fail: when a lock cannot be taken for a reason other than another process holding it (a file system without lock support such as some network mounts, no permission to create the lock file, a network error), writes warn once and proceed unlocked, and `lock_partition` warns and returns `TRUE` with attribute `locked = FALSE`. Cloud-synchronized folders lock on one computer only. Copying or archiving the array directory in the session that holds its locks releases them on `POSIX` systems. To lock anything else the same way, see `with_filelock`.

Options (each falls back to the environment variable, which worker processes inherit):

`filearray.lock` (`FILEARRAY_LOCK`) set to `FALSE` to stop writes from locking automatically; `lock_partition` still locks

`filearray.lock.timeout` (`FILEARRAY_LOCK_TIMEOUT`) seconds a write waits for a locked partition before it fails with an error of class `filearray_lock_timeout`; default `Inf`, wait until the partition is free. A value that is not a non-negative number warns once and is ignored

See Also

filearray, with_filelock

---

filearray_bind *Merge and bind homogeneous file arrays*

Description

The file arrays to be merged must be homogeneous: same data type, partition size, and partition length

Usage

```
filearray_bind(
  ...,
  .list = list(),
  filebase = tempfile(),
  symlink = FALSE,
  overwrite = FALSE,
  cache_ok = FALSE
)
```

Arguments

..., .list	file array instances
filebase	where to create merged array
symlink	whether to use file.symlink; if true, then partition files will be symbolic-linked to the original arrays, otherwise the partition files will be copied over. If you want your data to be portable, do not use symbolic-links. The default value is FALSE
overwrite	whether to overwrite when filebase already exists; default is false, which raises errors
cache_ok	see 'Details', only used if overwrite is true.

Details

The input arrays must share the same data type and partition size. The dimension for each partition should also be the same. For example an array x1 has dimension $100x20x30$ with partition size 1, then each partition dimension is $100x20x1$, and there are 30 partitions. x1 can bind with another array of the same partition size. This means if x2 has dimension $100x20x40$ and each partition size is 1, then x1 and x2 can be merged.

If filebase exists and overwrite is FALSE, an error will always raise. If overwrite=TRUE and cache_ok=FALSE, then the existing filebase will be erased and any data stored within will be lost. If both overwrite and cache_ok are TRUE, then , before erasing filebase, the function validates the existing array header and compare the header signatures. If the existing header signature is the

same as the array to be created, then the existing array will be returned. This `cache_ok` could be extremely useful when binding large arrays with `symlink=FALSE` as the cache might avoid moving files around. However, `cache_ok` should be enabled with caution. This is because only the header information will be compared, but the partition data will not be compared. If the existing array was generated from an old versions of the source arrays, but the data from the source arrays has been altered, then the `cache_ok=TRUE` is rarely proper as the cache is outdated.

The `symlink` option should be used with extra caution. Creating symbolic links is definitely faster than copying partition files. However, since the partition files are simply linked to the original partition files, changing to the input arrays will also affect the merged arrays, and vice versa; see 'Examples'. Also for arrays created from symbolic links, if the original arrays are deleted, while the merged arrays will not be invalidated, the corresponding partitions will no longer be accessible. Attempts to set deleted partitions will likely result in failure. Therefore `symlink` should be set to true when creating merged arrays are temporary for read-only purpose, and when speed and disk space is in consideration. For extended reading, please check `files` for details.

Value

A bound array in `'FileArray'` class.

Examples

```
partition_size <- 1
type <- "double"
x1 <- filearray_create(
    tempfile(), c(2,2), type = type,
    partition_size = partition_size)
x1[] <- 1:4
x2 <- filearray_create(
    tempfile(), c(2,1), type = type,
    partition_size = partition_size)
x2[] <- 5:6

y1 <- filearray_bind(x1, x2, symlink = FALSE)
y2 <- filearray_bind(x1, x2)

# y1 copies partition files, and y2 simply creates links
# if symlink is supported

y1[] - y2[]

# change x1
x1[1,1] <- NA

# y1 is not affected
y1[]

# y2 changes
y2[]
```

filearray_threads *Set or get file array threads*

Description

Will enable/disable multi-threaded reading or writing at C++ level.

Usage

```
filearray_threads(n, ...)
```

Arguments

n	number of threads to set. If n is negative, then default to the number of cores that computer has.
...	internally used

Value

An integer of current number of threads

fmap *Map multiple file arrays and save results*

Description

Advanced mapping function for multiple file arrays. fmap runs the mapping functions and stores the results in file arrays. fmap2 stores results in memory. This feature is experimental. There are several constraints to the input. Failure to meet these constraints may result in undefined results, or even crashes. Please read Section 'Details' carefully before using this function.

Usage

```
fmap(
  x,
  fun,
  .y = NULL,
  .buffer_count = NA_integer_,
  .output_size = NA_integer_,
  ...
)

fmap2(x, fun, .buffer_count = NA, .simplify = TRUE, ...)

fmap_element_wise(x, fun, .y, ..., .input_size = NA)
```

Arguments

x	a list of file arrays to map; each element of x must share the same dimensions.
fun	function that takes one list
.y	a file array object, used to save results
.buffer_count	number of total buffers (chunks) to run
.output_size	fun output vector length
...	other arguments passing to fun
.simplify	whether to apply simplify2array to the result
.input_size	number of elements to read from each array of x

Details

Denote the first argument of fun as input, The length of input equals the length of x. The size of each element of input is defined by .input_size, except for the last loop. For example, given dimension of each input array as $10x10x10x10$, if .input_size=100, then length(input[[1]])=100. The total number of runs equals to length(x[[1]])/100. If .input_size=300, then length(input[[1]]) will be 300 except for the last run. This is because 10000 cannot be divided by 300. The element length of the last run will be 100.

The returned variable length of fun will be checked by .output_size. If the output length exceed .output_size, an error will be raised.

Please make sure that length(.y)/length(x[[1]]) equals to .output_size/.input_size.

For fmap_element_wise, the input[[1]] and output length must be the consistent.

Value

File array instance .y

Examples

```
set.seed(1)
x1 <- filearray_create(tempfile(), dimension = c(100,20,3))
x1[] <- rnorm(6000)
x2 <- filearray_create(tempfile(), dimension = c(100,20,3))
x2[] <- rnorm(6000)

# Add two arrays
output <- filearray_create(tempfile(), dimension = c(100,20,3))
fmap(list(x1, x2), function(input) {
    input[[1]] + input[[2]]
}, output)

# check
range(output[] - (x1[] + x2[]))

output$delete()
```

```
# Calculate the maximum of x1/x2 for every 100 elements
# total 60 batches/loops (`.buffer_count`)
output <- filearray_create(tempfile(), dimension = c(20,3))
fmap(list(x1, x2), function(input) {
    max(input[[1]] / input[[2]])
}, .y = output, .buffer_count = 60)

# check
range(output[] - apply(x1[] / x2[], c(2,3), max))

output$delete()

# A large array example
if(interactive()) {
    x <- filearray_create(tempfile(), dimension = c(287, 100, 301, 4))
    dimnames(x) <- list(
        Trial = 1:287,
        Marker = 1:100,
        Time = 1:301,
        Location = 1:4
    )

    for (i in 1:4) {
        x[,,,i] <- runif(8638700)
    }
    # Step 1:
    # for each location, trial, and marker, calibrate (baseline)
    # according to first 50 time-points

    output <- filearray_create(tempfile(), dimension = dim(x))

    # baseline-percentage change
    fmap(
        list(x),
        function(input) {
            # get locational data
            location_data <- input[[1]]
            dim(location_data) <- c(287, 100, 301)

            # collapse over first 50 time points for
            # each trial, and marker
            baseline <- apply(location_data[,,1:50], c(1,2), mean)

            # calibrate
            calibrated <- sweep(location_data, c(1,2), baseline,
                                FUN = function(data, bl) {
                                    (data / bl - 1) * 100
                                })
            return(calibrated)
        },

        .y = output,
```

```
        # input dimension is 287 x 100 x 301 for each location
        # hence 4 loops in total
        .buffer_count = 4
    )

    # cleanup
    x$delete()

}

# cleanup
x1$delete()
x2$delete()
output$delete()
```

`fwhich` *A generic function of* `which` *that is* `'FileArray'` *compatible*

Description

A generic function of `which` that is `'FileArray'` compatible

Usage

```
fwhich(x, val, arr.ind = FALSE, ret.values = FALSE, ...)

## Default S3 method:
fwhich(x, val, arr.ind = FALSE, ret.values = FALSE, ...)

## S3 method for class 'FileArray'
fwhich(x, val, arr.ind = FALSE, ret.values = FALSE, ...)
```

Arguments

`x`	any R vector, matrix, array or file-array
`val`	values to find, or a function taking one argument (a slice of data vector) and returns either logical vector with the same length as the slice or index of the slice; see 'Examples'
`arr.ind`	logical; should array indices be returned when `x` is an array?
`ret.values`	whether to return the values of corresponding indices as an attributes; default is false
`...`	passed to `val` if `val` is a function

Value

The indices of `x` elements that are listed in `val`.

Examples

```
# ---- Default case ------------------------------------
x <- array(1:27 + 2, rep(3,3))

# find index of `x` equal to either 4 or 5
fwhich(x, c(4,5))
res <- fwhich(x, c(4,5), ret.values = TRUE)
res
attr(res, "values")

# ---- file-array case ---------------------------------
arr <- filearray_create(tempfile(), dim(x))
arr[] <- x
fwhich(arr, c(4,5))
fwhich(arr, c(4,5), arr.ind = TRUE, ret.values = TRUE)

arr[2:3, 1, 1]

# Clean up this example
arr$delete()

# ---- `val` is a function -----------------------------
x <- as_filearray(c(sample(15), 15), dimension = c(4,4))

ret <- fwhich(x, val = which.max,
              ret.values = TRUE, arr.ind = FALSE)

# ret is the index
ret == which.max(x[])

# attr(ret, "values") is the max value
max(x[]) == attr(ret, "values")

# customize `val`
fwhich(x, ret.values = TRUE, arr.ind = FALSE,
       val = function( slice ) {
           slice > 10 # or which(slice > 10)
       })
```

`mapreduce` *A map-reduce method to iterate blocks of file-array data with little memory usage*

Description

A map-reduce method to iterate blocks of file-array data with little memory usage

Usage

```
mapreduce(x, map, reduce, ...)

## S4 method for signature 'FileArray,ANY,function'
mapreduce(x, map, reduce, buffer_size = NA, ...)

## S4 method for signature 'FileArray,ANY,NULL'
mapreduce(x, map, reduce, buffer_size = NA, ...)

## S4 method for signature 'FileArray,ANY,missing'
mapreduce(x, map, reduce, buffer_size = NA, ...)
```

Arguments

x	a file array object
map	mapping function that receives 3 arguments; see 'Details'
reduce	NULL, or a function that takes a list as input
...	passed to other methods
buffer_size	control how we split the array; see 'Details'

Details

When handling out-of-memory arrays, it is recommended to load a block of array at a time and execute on block level. See apply for a implementation. When an array is too large, and when there are too many blocks, this operation will become very slow if computer memory is low. This is because the R will perform garbage collection frequently. Implemented in C++, mapreduce creates a buffer to store the block data. By reusing the memory over and over again, it is possible to iterate through the array with minimal garbage collections. Many statistics, including min, max, sum, mean, ... These statistics can be calculated in this way efficiently.

The function map contains three arguments: data (mandate), size (optional), and first_index (optional). The data is the buffer, whose length is consistent across iterations. size indicates the effective size of the buffer. If the partition size is not divisible by the buffer size, only first size elements of the data are from array, and the rest elements will be NA. This situation could only occurs when buffer_size is manually specified. By default, all of data should belong to arrays. The last argument first_index is the index of the first element data[1] in the whole array. It is useful when positional data is needed.

The buffer size, specified by buffer_size is an additional optional argument in .... Its default is NA, and will be calculated automatically. If manually specified, a large buffer size would be desired to speed up the calculation. The default buffer size will not exceed $nThreads x 2MB$, where nThreads is the number of threads set by filearray_threads. When partition length cannot be divided by the buffer size, instead of trimming the buffer, NAs will be filled to the buffer, passed to map function; see previous paragraph for treatments.

The function mapreduce ignores the missing partitions. That means if a partition is missing, its data will not be read nor passed to map function. Please run x$initialize_partition() to make sure partition files exist.

Value

If `reduce` is `NULL`, return mapped results, otherwise return reduced results from `reduce` function

Examples

```
x <- filearray_create(tempfile(), c(100, 100, 10))
x[] <- rnorm(1e5)

## calculate summation
# identical to sum(x[]), but is more feasible in large cases

mapreduce(x, map = function(data, size) {
    # make sure `data` is all from array
    if (length(data) != size) {
        data <- data[1:size]
    }
    sum(data)
}, reduce = function(mapped_list) {
    do.call(sum, mapped_list)
})


## Find elements are less than -3
positions <- mapreduce(
    x,
    map = function(data, size, first_index) {
        if (length(data) != size) {
            data <- data[1:size]
        }
        which(data < -3) + (first_index - 1)
    },
    reduce = function(mapped_list) {
        do.call(c, mapped_list)
    }
)

if(length(positions)) {
    x[[positions[1]]]
}
```

`S3-filearray` *'S3' methods for 'FileArray'*

Description

These are 'S3' methods for 'FileArray'

Usage

```
## S3 method for class 'FileArray'

  x[
  i,
  ...,
  drop = TRUE,
  reshape = NULL,
  strict = TRUE,
  dimnames = TRUE,
  split_dim = 0
]

## S3 replacement method for class 'FileArray'
x[i, ..., lazy = FALSE] <- value

## S3 method for class 'FileArray'
x[[i]]

## S3 method for class 'FileArray'
as.array(x, reshape = NULL, drop = FALSE, ...)

## S3 method for class 'FileArray'
dim(x)

## S3 method for class 'FileArray'
dimnames(x)

## S3 replacement method for class 'FileArray'
dimnames(x) <- value

## S3 method for class 'FileArray'
length(x)

## S3 method for class 'FileArray'
max(x, na.rm = FALSE, ...)

## S3 method for class 'FileArray'
min(x, na.rm = FALSE, ...)

## S3 method for class 'FileArray'
range(x, na.rm = FALSE, ...)

## S3 method for class 'FileArray'
sum(x, na.rm = FALSE, ...)

## S3 method for class 'FileArray'
subset(x, ..., drop = FALSE, .env = parent.frame())
```

Arguments

x	a file array
i, ...	index set, or passed to other methods
drop	whether to drop dimensions; see topic Extract
reshape	a new dimension to set before returning subset results; default is NULL (use default dimensions)
strict	whether to allow indices to exceed bound; currently only accept TRUE
dimnames	whether to preserve dimnames
split_dim	internally used; split dimension and calculate indices to manually speed up the subset; value ranged from 0 to size of dimension minus one.
lazy	whether to lazy-evaluate the method, only works when assigning arrays with logical array index
value	value to substitute or set
na.rm	whether to remove NA values during the calculation
.env	environment to evaluate formula when evaluating subset margin indices.

Functions

- FileArray[]: subset array
- FileArray[] <- value: subset assign array
- FileArray[[]]: get element by index
- as.array(FileArray): converts file array to native array in R
- dim(FileArray): get dimensions
- dimnames(FileArray): get dimension names
- dimnames(FileArray) <- value: set dimension names
- length(FileArray): get array length
- max(FileArray): get max value
- min(FileArray): get min value
- range(FileArray): get value range
- sum(FileArray): get summation
- subset(FileArray): get subset file array with formulae

S4-filearray *'S4' methods for* FileArray

Description

'S4' methods for FileArray

Usage

```
## S4 method for signature 'FileArray,FileArray'
e1 + e2

## S4 method for signature 'FileArray,numeric'
e1 + e2

## S4 method for signature 'numeric,FileArray'
e1 + e2

## S4 method for signature 'FileArray,complex'
e1 + e2

## S4 method for signature 'complex,FileArray'
e1 + e2

## S4 method for signature 'FileArray,logical'
e1 + e2

## S4 method for signature 'logical,FileArray'
e1 + e2

## S4 method for signature 'FileArray,array'
e1 + e2

## S4 method for signature 'array,FileArray'
e1 + e2

## S4 method for signature 'FileArray,FileArray'
e1 - e2

## S4 method for signature 'FileArray,numeric'
e1 - e2

## S4 method for signature 'numeric,FileArray'
e1 - e2

## S4 method for signature 'FileArray,complex'
e1 - e2
```

```
## S4 method for signature 'complex,FileArray'
e1 - e2

## S4 method for signature 'FileArray,logical'
e1 - e2

## S4 method for signature 'logical,FileArray'
e1 - e2

## S4 method for signature 'FileArray,array'
e1 - e2

## S4 method for signature 'array,FileArray'
e1 - e2

## S4 method for signature 'FileArray,FileArray'
e1 * e2

## S4 method for signature 'FileArray,numeric'
e1 * e2

## S4 method for signature 'numeric,FileArray'
e1 * e2

## S4 method for signature 'FileArray,complex'
e1 * e2

## S4 method for signature 'complex,FileArray'
e1 * e2

## S4 method for signature 'FileArray,logical'
e1 * e2

## S4 method for signature 'logical,FileArray'
e1 * e2

## S4 method for signature 'FileArray,array'
e1 * e2

## S4 method for signature 'array,FileArray'
e1 * e2

## S4 method for signature 'FileArray,FileArray'
e1 / e2

## S4 method for signature 'FileArray,numeric'
e1 / e2
```

```
## S4 method for signature 'numeric,FileArray'
e1 / e2

## S4 method for signature 'FileArray,complex'
e1 / e2

## S4 method for signature 'complex,FileArray'
e1 / e2

## S4 method for signature 'FileArray,logical'
e1 / e2

## S4 method for signature 'logical,FileArray'
e1 / e2

## S4 method for signature 'FileArray,array'
e1 / e2

## S4 method for signature 'array,FileArray'
e1 / e2

## S4 method for signature 'FileArray,FileArray'
e1 ^ e2

## S4 method for signature 'FileArray,numeric'
e1 ^ e2

## S4 method for signature 'numeric,FileArray'
e1 ^ e2

## S4 method for signature 'FileArray,complex'
e1 ^ e2

## S4 method for signature 'complex,FileArray'
e1 ^ e2

## S4 method for signature 'FileArray,logical'
e1 ^ e2

## S4 method for signature 'logical,FileArray'
e1 ^ e2

## S4 method for signature 'FileArray,array'
e1 ^ e2

## S4 method for signature 'array,FileArray'
e1 ^ e2
```

```
## S4 method for signature 'FileArray,FileArray'
e1 %% e2

## S4 method for signature 'FileArray,numeric'
e1 %% e2

## S4 method for signature 'numeric,FileArray'
e1 %% e2

## S4 method for signature 'FileArray,complex'
e1 %% e2

## S4 method for signature 'complex,FileArray'
e1 %% e2

## S4 method for signature 'FileArray,logical'
e1 %% e2

## S4 method for signature 'logical,FileArray'
e1 %% e2

## S4 method for signature 'FileArray,array'
e1 %% e2

## S4 method for signature 'array,FileArray'
e1 %% e2

## S4 method for signature 'FileArray,FileArray'
e1 %/% e2

## S4 method for signature 'FileArray,numeric'
e1 %/% e2

## S4 method for signature 'numeric,FileArray'
e1 %/% e2

## S4 method for signature 'FileArray,complex'
e1 %/% e2

## S4 method for signature 'complex,FileArray'
e1 %/% e2

## S4 method for signature 'FileArray,logical'
e1 %/% e2

## S4 method for signature 'logical,FileArray'
e1 %/% e2
```

```
## S4 method for signature 'FileArray,array'
e1 %/% e2

## S4 method for signature 'array,FileArray'
e1 %/% e2

## S4 method for signature 'FileArray,FileArray'
e1 == e2

## S4 method for signature 'FileArray,numeric'
e1 == e2

## S4 method for signature 'numeric,FileArray'
e1 == e2

## S4 method for signature 'FileArray,complex'
e1 == e2

## S4 method for signature 'complex,FileArray'
e1 == e2

## S4 method for signature 'FileArray,logical'
e1 == e2

## S4 method for signature 'logical,FileArray'
e1 == e2

## S4 method for signature 'FileArray,array'
e1 == e2

## S4 method for signature 'array,FileArray'
e1 == e2

## S4 method for signature 'FileArray,FileArray'
e1 > e2

## S4 method for signature 'FileArray,numeric'
e1 > e2

## S4 method for signature 'numeric,FileArray'
e1 > e2

## S4 method for signature 'FileArray,complex'
e1 > e2

## S4 method for signature 'complex,FileArray'
e1 > e2
```

```
## S4 method for signature 'FileArray,logical'
e1 > e2

## S4 method for signature 'logical,FileArray'
e1 > e2

## S4 method for signature 'FileArray,array'
e1 > e2

## S4 method for signature 'array,FileArray'
e1 > e2

## S4 method for signature 'FileArray,FileArray'
e1 < e2

## S4 method for signature 'FileArray,numeric'
e1 < e2

## S4 method for signature 'numeric,FileArray'
e1 < e2

## S4 method for signature 'FileArray,complex'
e1 < e2

## S4 method for signature 'complex,FileArray'
e1 < e2

## S4 method for signature 'FileArray,logical'
e1 < e2

## S4 method for signature 'logical,FileArray'
e1 < e2

## S4 method for signature 'FileArray,array'
e1 < e2

## S4 method for signature 'array,FileArray'
e1 < e2

## S4 method for signature 'FileArray,FileArray'
e1 != e2

## S4 method for signature 'FileArray,numeric'
e1 != e2

## S4 method for signature 'numeric,FileArray'
e1 != e2
```

```
## S4 method for signature 'FileArray,complex'
e1 != e2

## S4 method for signature 'complex,FileArray'
e1 != e2

## S4 method for signature 'FileArray,logical'
e1 != e2

## S4 method for signature 'logical,FileArray'
e1 != e2

## S4 method for signature 'FileArray,array'
e1 != e2

## S4 method for signature 'array,FileArray'
e1 != e2

## S4 method for signature 'FileArray,FileArray'
e1 >= e2

## S4 method for signature 'FileArray,numeric'
e1 >= e2

## S4 method for signature 'numeric,FileArray'
e1 >= e2

## S4 method for signature 'FileArray,complex'
e1 >= e2

## S4 method for signature 'complex,FileArray'
e1 >= e2

## S4 method for signature 'FileArray,logical'
e1 >= e2

## S4 method for signature 'logical,FileArray'
e1 >= e2

## S4 method for signature 'FileArray,array'
e1 >= e2

## S4 method for signature 'array,FileArray'
e1 >= e2

## S4 method for signature 'FileArray,FileArray'
e1 <= e2
```

```
## S4 method for signature 'FileArray,numeric'
e1 <= e2

## S4 method for signature 'numeric,FileArray'
e1 <= e2

## S4 method for signature 'FileArray,complex'
e1 <= e2

## S4 method for signature 'complex,FileArray'
e1 <= e2

## S4 method for signature 'FileArray,logical'
e1 <= e2

## S4 method for signature 'logical,FileArray'
e1 <= e2

## S4 method for signature 'FileArray,array'
e1 <= e2

## S4 method for signature 'array,FileArray'
e1 <= e2

## S4 method for signature 'FileArray,FileArray'
e1 & e2

## S4 method for signature 'FileArray,numeric'
e1 & e2

## S4 method for signature 'numeric,FileArray'
e1 & e2

## S4 method for signature 'FileArray,complex'
e1 & e2

## S4 method for signature 'complex,FileArray'
e1 & e2

## S4 method for signature 'FileArray,logical'
e1 & e2

## S4 method for signature 'logical,FileArray'
e1 & e2

## S4 method for signature 'FileArray,array'
e1 & e2
```

```
## S4 method for signature 'array,FileArray'
e1 & e2

## S4 method for signature 'FileArray,FileArray'
e1 | e2

## S4 method for signature 'FileArray,numeric'
e1 | e2

## S4 method for signature 'numeric,FileArray'
e1 | e2

## S4 method for signature 'FileArray,complex'
e1 | e2

## S4 method for signature 'complex,FileArray'
e1 | e2

## S4 method for signature 'FileArray,logical'
e1 | e2

## S4 method for signature 'logical,FileArray'
e1 | e2

## S4 method for signature 'FileArray,array'
e1 | e2

## S4 method for signature 'array,FileArray'
e1 | e2

## S4 method for signature 'FileArray'
!x

## S4 method for signature 'FileArray'
exp(x)

## S4 method for signature 'FileArray'
expm1(x)

## S4 method for signature 'FileArray'
log(x, base = exp(1))

## S4 method for signature 'FileArray'
log10(x)

## S4 method for signature 'FileArray'
log2(x)
```

```
## S4 method for signature 'FileArray'
log1p(x)

## S4 method for signature 'FileArray'
abs(x)

## S4 method for signature 'FileArray'
sqrt(x)

## S4 method for signature 'FileArray'
sign(x)

## S4 method for signature 'FileArray'
signif(x, digits = 6)

## S4 method for signature 'FileArray'
trunc(x, ...)

## S4 method for signature 'FileArray'
floor(x)

## S4 method for signature 'FileArray'
ceiling(x)

## S4 method for signature 'FileArray'
round(x, digits = 0)

## S4 method for signature 'FileArray'
acos(x)

## S4 method for signature 'FileArray'
acosh(x)

## S4 method for signature 'FileArray'
asin(x)

## S4 method for signature 'FileArray'
asinh(x)

## S4 method for signature 'FileArray'
atan(x)

## S4 method for signature 'FileArray'
atanh(x)

## S4 method for signature 'FileArray'
cos(x)
```

```
## S4 method for signature 'FileArray'
cosh(x)

## S4 method for signature 'FileArray'
cospi(x)

## S4 method for signature 'FileArray'
sin(x)

## S4 method for signature 'FileArray'
sinh(x)

## S4 method for signature 'FileArray'
sinpi(x)

## S4 method for signature 'FileArray'
tan(x)

## S4 method for signature 'FileArray'
tanh(x)

## S4 method for signature 'FileArray'
tanpi(x)

## S4 method for signature 'FileArray'
gamma(x)

## S4 method for signature 'FileArray'
lgamma(x)

## S4 method for signature 'FileArray'
digamma(x)

## S4 method for signature 'FileArray'
trigamma(x)

## S4 method for signature 'FileArray'
Arg(z)

## S4 method for signature 'FileArray'
Conj(z)

## S4 method for signature 'FileArray'
Im(z)

## S4 method for signature 'FileArray'
Mod(z)
```

```
## S4 method for signature 'FileArray'
Re(z)

## S4 method for signature 'FileArray'
is.na(x)
```

Arguments

x, z, e1, e2	FileArray or compatible data
base, digits, ...	passed to other methods

Value

See S4groupGeneric

---

typeof *The type of a file array (extended)*

Description

The type of a file array (extended)

Usage

```
typeof(x)

## S4 method for signature 'FileArray'
typeof(x)

## S4 method for signature 'FileArrayProxy'
typeof(x)
```

Arguments

x	any file array

Value

A character string. The possible values are "double", "integer", "logical", and "raw"

`with_filelock` *Evaluate an expression while holding a file lock*

Description

Runs `expr` while holding an exclusive lock on `lockfile`, so that only one R session at a time runs code under that lock; other sessions asking for it wait until it is released. The lock has nothing to do with file arrays: any package or script can use it.

Usage

```
with_filelock(
  expr,
  lockfile,
  timeout = Inf,
  on_unsupported = c("warn", "error", "ignore")
)
```

Arguments

`expr`	expression to evaluate while the lock is held; it is evaluated in the calling environment
`lockfile`	path of the lock file, a file used only for locking. It is created if missing (its directory must exist), never written to, and left in place afterwards
`timeout`	seconds to wait for another session to release the lock: `Inf` (default) waits until the lock is free, `0` tries once. After the time-out an error of class `filearray_lock_timeout` is raised and `expr` is not evaluated
`on_unsupported`	what to do when the lock cannot be taken for a reason other than another session holding it, for example a file system without lock support or a lock file that cannot be created: `"warn"` (default) warns once per lock file and session, then evaluates `expr` without the lock; `"error"` raises an error and does not evaluate `expr`; `"ignore"` evaluates `expr` without the lock and without a warning. The warning and the error have class `filearray_lock_unsupported`

Details

The lock is an operating-system lock on the first byte of `lockfile` (`fcntl` on `POSIX` systems, `LockFileEx` on 'Windows'), so it is released when a session ends, even when it crashes. It is the lock that `lock()` of the **filelock** package takes in exclusive mode, so the two exclude each other between sessions.

Locks belong to the R session: a nested `with_filelock()` call on the same lock file in the same session runs at once instead of waiting for itself. Code running in the same session is therefore never excluded, even when it serves different users, as several 'Shiny' sessions in one R process do.

Use a dedicated lock file, and do not open it in `expr`: on `POSIX` systems closing any connection to the lock file releases the lock, and on 'Windows' the locked byte cannot be read or written.

The lock is advisory: it only excludes code that locks the same file. Forked workers (such as those of `mclapply` in **parallel**) do not inherit it. Sessions that nest locks on two files in opposite orders wait for each other until their time-outs. On network file systems the lock works across computers only if the file system supports locks; folders synchronized by cloud services are locked on one computer only.

Value

The value of `expr`

Examples

```
lockfile <- tempfile(fileext = ".lock")
log <- tempfile()

# one R session at a time appends to the log
with_filelock({
    cat("one line\n", file = log, append = TRUE)
}, lockfile = lockfile)

readLines(log)

unlink(c(lockfile, log))
```

Index

!,FileArray-method *(S4-filearray)*, 19
!=,FileArray,FileArray-method *(S4-filearray)*, 19
!=,FileArray,array-method *(S4-filearray)*, 19
!=,FileArray,complex-method *(S4-filearray)*, 19
!=,FileArray,logical-method *(S4-filearray)*, 19
!=,FileArray,numeric-method *(S4-filearray)*, 19
!=,array,FileArray-method *(S4-filearray)*, 19
!=,complex,FileArray-method *(S4-filearray)*, 19
!=,logical,FileArray-method *(S4-filearray)*, 19
!=,numeric,FileArray-method *(S4-filearray)*, 19
*,FileArray,FileArray-method *(S4-filearray)*, 19
*,FileArray,array-method *(S4-filearray)*, 19
*,FileArray,complex-method *(S4-filearray)*, 19
*,FileArray,logical-method *(S4-filearray)*, 19
*,FileArray,numeric-method *(S4-filearray)*, 19
*,array,FileArray-method *(S4-filearray)*, 19
*,complex,FileArray-method *(S4-filearray)*, 19
*,logical,FileArray-method *(S4-filearray)*, 19
*,numeric,FileArray-method *(S4-filearray)*, 19
+,FileArray,FileArray-method *(S4-filearray)*, 19
+,FileArray,array-method *(S4-filearray)*, 19
+,FileArray,complex-method *(S4-filearray)*, 19
+,FileArray,logical-method *(S4-filearray)*, 19
+,FileArray,numeric-method *(S4-filearray)*, 19
+,array,FileArray-method *(S4-filearray)*, 19
+,complex,FileArray-method *(S4-filearray)*, 19
+,logical,FileArray-method *(S4-filearray)*, 19
+,numeric,FileArray-method *(S4-filearray)*, 19
-,FileArray,FileArray-method *(S4-filearray)*, 19
-,FileArray,array-method *(S4-filearray)*, 19
-,FileArray,complex-method *(S4-filearray)*, 19
-,FileArray,logical-method *(S4-filearray)*, 19
-,FileArray,numeric-method *(S4-filearray)*, 19
-,array,FileArray-method *(S4-filearray)*, 19
-,complex,FileArray-method *(S4-filearray)*, 19
-,logical,FileArray-method *(S4-filearray)*, 19
-,numeric,FileArray-method *(S4-filearray)*, 19
/,FileArray,FileArray-method *(S4-filearray)*, 19
/,FileArray,array-method *(S4-filearray)*, 19
/,FileArray,complex-method

(S4-filearray), 19
/,FileArray,logical-method
(S4-filearray), 19
/,FileArray,numeric-method
(S4-filearray), 19
/,array,FileArray-method
(S4-filearray), 19
/,complex,FileArray-method
(S4-filearray), 19
/,logical,FileArray-method
(S4-filearray), 19
/,numeric,FileArray-method
(S4-filearray), 19
<,FileArray,FileArray-method
(S4-filearray), 19
<,FileArray,array-method
(S4-filearray), 19
<,FileArray,complex-method
(S4-filearray), 19
<,FileArray,logical-method
(S4-filearray), 19
<,FileArray,numeric-method
(S4-filearray), 19
<,array,FileArray-method
(S4-filearray), 19
<,complex,FileArray-method
(S4-filearray), 19
<,logical,FileArray-method
(S4-filearray), 19
<,numeric,FileArray-method
(S4-filearray), 19
<=,FileArray,FileArray-method
(S4-filearray), 19
<=,FileArray,array-method
(S4-filearray), 19
<=,FileArray,complex-method
(S4-filearray), 19
<=,FileArray,logical-method
(S4-filearray), 19
<=,FileArray,numeric-method
(S4-filearray), 19
<=,array,FileArray-method
(S4-filearray), 19
<=,complex,FileArray-method
(S4-filearray), 19
<=,logical,FileArray-method
(S4-filearray), 19
<=,numeric,FileArray-method
(S4-filearray), 19
==,FileArray,FileArray-method
(S4-filearray), 19
==,FileArray,array-method
(S4-filearray), 19
==,FileArray,complex-method
(S4-filearray), 19
==,FileArray,logical-method
(S4-filearray), 19
==,FileArray,numeric-method
(S4-filearray), 19
==,array,FileArray-method
(S4-filearray), 19
==,complex,FileArray-method
(S4-filearray), 19
==,logical,FileArray-method
(S4-filearray), 19
==,numeric,FileArray-method
(S4-filearray), 19
>,FileArray,FileArray-method
(S4-filearray), 19
>,FileArray,array-method
(S4-filearray), 19
>,FileArray,complex-method
(S4-filearray), 19
>,FileArray,logical-method
(S4-filearray), 19
>,FileArray,numeric-method
(S4-filearray), 19
>,array,FileArray-method
(S4-filearray), 19
>,complex,FileArray-method
(S4-filearray), 19
>,logical,FileArray-method
(S4-filearray), 19
>,numeric,FileArray-method
(S4-filearray), 19
>=,FileArray,FileArray-method
(S4-filearray), 19
>=,FileArray,array-method
(S4-filearray), 19
>=,FileArray,complex-method
(S4-filearray), 19
>=,FileArray,logical-method
(S4-filearray), 19
>=,FileArray,numeric-method
(S4-filearray), 19
>=,array,FileArray-method

(S4-filearray), 19
>=,complex,FileArray-method
(S4-filearray), 19
>=,logical,FileArray-method
(S4-filearray), 19
>=,numeric,FileArray-method
(S4-filearray), 19
[.FileArray (S3-filearray), 16
[<-.FileArray (S3-filearray), 16
[[.FileArray (S3-filearray), 16
%/%,FileArray,FileArray-method
(S4-filearray), 19
%/%,FileArray,array-method
(S4-filearray), 19
%/%,FileArray,complex-method
(S4-filearray), 19
%/%,FileArray,logical-method
(S4-filearray), 19
%/%,FileArray,numeric-method
(S4-filearray), 19
%/%,array,FileArray-method
(S4-filearray), 19
%/%,complex,FileArray-method
(S4-filearray), 19
%/%,logical,FileArray-method
(S4-filearray), 19
%/%,numeric,FileArray-method
(S4-filearray), 19
%%,FileArray,FileArray-method
(S4-filearray), 19
%%,FileArray,array-method
(S4-filearray), 19
%%,FileArray,complex-method
(S4-filearray), 19
%%,FileArray,logical-method
(S4-filearray), 19
%%,FileArray,numeric-method
(S4-filearray), 19
%%,array,FileArray-method
(S4-filearray), 19
%%,complex,FileArray-method
(S4-filearray), 19
%%,logical,FileArray-method
(S4-filearray), 19
%%,numeric,FileArray-method
(S4-filearray), 19
&,FileArray,FileArray-method
(S4-filearray), 19
&,FileArray,array-method
(S4-filearray), 19
&,FileArray,complex-method
(S4-filearray), 19
&,FileArray,logical-method
(S4-filearray), 19
&,FileArray,numeric-method
(S4-filearray), 19
&,array,FileArray-method
(S4-filearray), 19
&,complex,FileArray-method
(S4-filearray), 19
&,logical,FileArray-method
(S4-filearray), 19
&,numeric,FileArray-method
(S4-filearray), 19
^,FileArray,FileArray-method
(S4-filearray), 19
^,FileArray,array-method
(S4-filearray), 19
^,FileArray,complex-method
(S4-filearray), 19
^,FileArray,logical-method
(S4-filearray), 19
^,FileArray,numeric-method
(S4-filearray), 19
^,array,FileArray-method
(S4-filearray), 19
^,complex,FileArray-method
(S4-filearray), 19
^,logical,FileArray-method
(S4-filearray), 19
^,numeric,FileArray-method
(S4-filearray), 19

abs,FileArray-method (S4-filearray), 19
acos,FileArray-method (S4-filearray), 19
acosh,FileArray-method (S4-filearray), 19
apply, 2, 2, *15*
apply,FileArray-method (apply), 2
apply,FileArrayProxy-method (apply), 2
Arg,FileArray-method (S4-filearray), 19
as.array.FileArray (S3-filearray), 16
as_filearray, 3
as_filearrayproxy (as_filearray), 3
asin,FileArray-method (S4-filearray), 19
asinh,FileArray-method (S4-filearray), 19

atan,FileArray-method *(S4-filearray)*, 19
atanh,FileArray-method *(S4-filearray)*, 19

ceiling,FileArray-method *(S4-filearray)*, 19
Conj,FileArray-method *(S4-filearray)*, 19
cos,FileArray-method *(S4-filearray)*, 19
cosh,FileArray-method *(S4-filearray)*, 19
cospi,FileArray-method *(S4-filearray)*, 19

digamma,FileArray-method *(S4-filearray)*, 19
dim.FileArray *(S3-filearray)*, 16
dimnames, *18*
dimnames.FileArray *(S3-filearray)*, 16
dimnames<-.FileArray *(S3-filearray)*, 16

exp,FileArray-method *(S4-filearray)*, 19
expm1,FileArray-method *(S4-filearray)*, 19
Extract, *18*

file.symlink, *8*
filearray, *7*, *8*
filearray *(as_filearray)*, 3
FileArray-class, 6
filearray_bind, *4*, *7*, 8
filearray_checkload *(as_filearray)*, 3
filearray_create, *6*
filearray_create *(as_filearray)*, 3
filearray_load, *6*
filearray_load *(as_filearray)*, 3
filearray_load_or_create *(as_filearray)*, 3
filearray_threads, 10, *15*
FileArrayProxy *(as_filearray)*, 3
FileArrayProxy-class *(as_filearray)*, 3
files, *9*
floor,FileArray-method *(S4-filearray)*, 19
fmap, *7*, 10
fmap2 *(fmap)*, 10
fmap_element_wise, *7*
fmap_element_wise *(fmap)*, 10
fwhich, 13

gamma,FileArray-method *(S4-filearray)*, 19

Im,FileArray-method *(S4-filearray)*, 19
is.na,FileArray-method *(S4-filearray)*, 19

length.FileArray *(S3-filearray)*, 16
lgamma,FileArray-method *(S4-filearray)*, 19
log,FileArray-method *(S4-filearray)*, 19
log10,FileArray-method *(S4-filearray)*, 19
log1p,FileArray-method *(S4-filearray)*, 19
log2,FileArray-method *(S4-filearray)*, 19

mapreduce, 14
mapreduce,FileArray,ANY,function-method *(mapreduce)*, 14
mapreduce,FileArray,ANY,missing-method *(mapreduce)*, 14
mapreduce,FileArray,ANY,NULL-method *(mapreduce)*, 14
max.FileArray *(S3-filearray)*, 16
min.FileArray *(S3-filearray)*, 16
Mod,FileArray-method *(S4-filearray)*, 19

range.FileArray *(S3-filearray)*, 16
Re,FileArray-method *(S4-filearray)*, 19
round,FileArray-method *(S4-filearray)*, 19

S3-filearray, 16
S4-filearray, 19
S4groupGeneric, *30*
sign,FileArray-method *(S4-filearray)*, 19
signif,FileArray-method *(S4-filearray)*, 19
simplify2array, *11*
sin,FileArray-method *(S4-filearray)*, 19
sinh,FileArray-method *(S4-filearray)*, 19
sinpi,FileArray-method *(S4-filearray)*, 19
sqrt,FileArray-method *(S4-filearray)*, 19
subset.FileArray *(S3-filearray)*, 16
sum.FileArray *(S3-filearray)*, 16

tan,FileArray-method *(S4-filearray)*, 19
tanh,FileArray-method *(S4-filearray)*, 19
tanpi,FileArray-method *(S4-filearray)*, 19

trigamma,FileArray-method
(S4-filearray), 19
trunc,FileArray-method (S4-filearray),
19
typeof, 30
typeof,FileArray-method (typeof), 30
typeof,FileArrayProxy-method (typeof),
30

with_filelock, *7*, *8*, 31